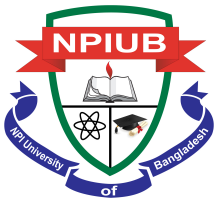


NPI UNIVERSITY OF BANGLADESH

Manikganj, Dhaka, Bangladesh

UniBus Live: A Real-Time Bilingual University Bus Tracking System with Resilient Dual-Source GPS Acquisition

A thesis submitted to the Department of Computer Science and Engineering,
NPI University of Bangladesh, in partial fulfillment of the requirements
for the degree of

Bachelor of Science in Computer Science and Engineering

Submitted by

Md. Shishir Hossain ID: 0852220005101005
Md. Sajib Hasan ID: 0852220005101008
Md. Showkat Hossen ID: 0852220005101009

Batch: 12th

Supervised by

Mahbubur Rahman Chowdhury
Senior Lecturer
Department of Computer Science and Engineering
NPI University of Bangladesh

July 2026

Declaration

We hereby declare that the thesis titled *"UniBus Live: A Real-Time Bilingual University Bus Tracking System with Resilient Dual-Source GPS Acquisition"* is the outcome of our own work, carried out under the supervision of Mahbubur Rahman Chowdhury, Senior Lecturer, Department of Computer Science and Engineering, NPI University of Bangladesh. The system described here was designed, implemented, deployed, and evaluated by us, and the results reported in this thesis were obtained from the operational data of our own pilot deployment.

We further declare that neither this thesis nor any part of it has been submitted elsewhere for the award of any degree or diploma, and that all external sources of information used in this work have been duly acknowledged and cited.

Md. Shishir Hossain
ID: 0852220005101005, Batch: 12th

Md. Sajib Hasan
ID: 0852220005101008, Batch: 12th

Md. Showkat Hossen
ID: 0852220005101009, Batch: 12th

Approval

The thesis titled *"UniBus Live: A Real-Time Bilingual University Bus Tracking System with Resilient Dual-Source GPS Acquisition"*, submitted by Md. Shishir Hossain (ID: 0852220005101005), Md. Sajib Hasan (ID: 0852220005101008), and Md. Showkat Hossen (ID: 0852220005101009) of the 12th batch, Department of Computer Science and Engineering, NPI University of Bangladesh, has been accepted as satisfactory in partial fulfillment of the requirements for the degree of **Bachelor of Science in Computer Science and Engineering** and approved as to its style and contents.

Supervisor

. . . . . .

Mahbubur Rahman Chowdhury
Senior Lecturer
Department of Computer Science and Engineering
NPI University of Bangladesh

Head of the Department

. . . . . .

Kartik Chandra Mandal
Associate Professor and Head of the Department
Department of Computer Science and Engineering
NPI University of Bangladesh

External Expert Member

. . . . . .

Prof. Dr. Md. Musfique Anwar
Department of Computer Science and Engineering
Jahangirnagar University
Savar, Dhaka

Acknowledgement

First and foremost, we express our deepest gratitude to the Almighty for giving us the strength, patience, and perseverance to complete this work.

We are sincerely grateful to our supervisor, **Mahbubur Rahman Chowdhury**, Senior Lecturer, Department of Computer Science and Engineering, NPI University of Bangladesh, for his continuous guidance, constructive criticism, and encouragement at every stage of this thesis. His insistence on evaluating the system against real operational data, rather than simulations, shaped the character of this work.

We thank the **Department of Computer Science and Engineering, NPI University of Bangladesh**, and its faculty members for providing the academic foundation on which this project was built, and the university transport administration for permitting us to install a GPS tracking device on an operational bus and to run a seven-week live pilot on the real university fleet.

We are also grateful to the drivers who cooperated with the smartphone fallback trials, and to the thirty-eight fellow students who registered as pilot passengers, used the application on their daily commutes, and gave us the feedback that drove many of the refinements described in this thesis.

Finally, we thank our families for their unconditional support throughout our undergraduate studies.

The Authors
July 2026

Abstract

University bus services carry thousands of students every day, yet at many institutions in developing regions the riders have no way of knowing where a bus actually is: a student standing at a stop cannot tell whether the bus departed five minutes ago or is still fifteen minutes away. The consequences are missed buses, long unsheltered waits, and avoidable crowding. Commercial fleet-tracking products exist, but they are costly, oriented toward logistics rather than passengers, and rarely available in the local language.

This thesis presents *UniBus Live*, a complete, deployed, low-cost real-time bus tracking platform built for university transport. The system consists of a bilingual (English and Bangla) passenger mobile application, a web-based administration console, and a central application server backed by a 31-model relational database, all hosted on a single low-cost virtual private server. Vehicle positions are acquired from two independent sources: a fixed, vehicle-grade GPS/GSM tracker that operates fully automatically as the primary source—requiring no driver action, device, or data connection—and the driver's smartphone as an optional automatic fallback, reconciled per fix by a health-based source arbitration component. Incoming fixes pass through a quality-filtering pipeline before a lightweight geometric estimator projects the position onto the route polyline and computes a per-stop arrival time with a confidence label. Updates are pushed to subscribed passengers over WebSocket within seconds.

The system was evaluated in a seven-week pilot (6 May–23 June 2026) on the operational university fleet, covering 9 routes, 3 buses, 38 registered passengers, and 71 completed trips, during which 46,548 raw GPS reports were ingested. The median position-acquisition latency was 5.9 s (mean 7.4 s), and the dual-source design was exercised in practice: 64% of trips were tracked by the hardware tracker and 36% by the smartphone fallback, largely reflecting the two buses not yet fitted with trackers. The pilot also revealed that roughly 98% of raw device reports were redundant transmissions made while buses were parked; a two-sided optimization—device-side reporting reconfiguration (5 s moving / 300 s parked) and server-side trip-aware polling throttling (8 s active / 60 s idle)—cut idle-period reporting by up to 60$\times$ without affecting live tracking. Of the arrival and service notifications delivered to riders during the pilot, 80% were read. These results demonstrate that resilient, passenger-centric, bilingual real-time bus tracking is feasible within a student-project budget.

Keywords: real-time systems, GPS tracking, public transport, estimated time of arrival, WebSocket, mobile application, intelligent transportation systems

Contents

Declaration i

Approval ii

Acknowledgement iii

Abstract iv

List of Abbreviations xi

1 Introduction 1

1.1 Background . . . 1
1.2 Motivation . . . 1
1.3 Problem Statement . . . 2
1.4 Objectives . . . 3
1.5 Scope of the Work . . . 3
1.6 Contributions . . . 4
1.7 Organization of the Thesis . . . 4

2 Literature Review 6

2.1 Overview . . . 6
2.2 Campus and Real-Time Bus Tracking Systems . . . 6
2.3 Smartphone-Based and Crowdsourced Positioning . . . 7
2.4 Bus Arrival-Time Prediction . . . 7
2.5 GPS Quality, Filtering, and Map Matching . . . 8
2.6 Open Telematics Platforms . . . 8
2.7 Research Gap and Positioning of This Work . . . 9
2.8 Summary . . . 9

3 System Analysis and Requirements 10

3.1 Stakeholder Analysis . . . 10
3.2 Functional Requirements . . . 10
3.3 Non-Functional Requirements . . . 12
3.4 Feasibility Study . . . 12
3.4.1 Technical Feasibility . . . 12

3.4.2 Economic Feasibility 13
3.4.3 Operational Feasibility 13
3.5 Development Methodology 13
3.6 Tools and Technologies 13
3.7 Summary 14

4 System Design **15**
4.1 Architectural Overview 15
4.2 Real-Time Data Flow 17
4.3 GPS-Acquisition Layer 17
4.3.1 Primary Source: Fixed Vehicle Tracker 17
4.3.2 Secondary Source: Driver Smartphone (Optional) 18
4.3.3 Ingesting from the Telematics Gateway 18
4.4 Source Arbitration Design 18
4.5 Location-Quality Filtering Design 19
4.6 Arrival-Time Estimation Design 19
4.7 Trip Lifecycle Design 20
4.8 Database Design 21
4.9 Realtime Delivery Design 21
4.10 Security Design 22
4.11 Summary 22

5 Implementation **23**
5.1 Backend Application Server 23
5.2 Traccar Integration 23
5.3 Source Arbitration and Filtering 24
5.4 ETA Computation 24
5.5 Trip Lifecycle and Background Jobs 24
5.6 Passenger Mobile Application 25
5.7 Driver Mobile Application (Fallback Publisher) 26
5.8 Administration Console 27
5.9 On-Vehicle Device Configuration 28
5.10 Deployment 29
5.11 Summary 29

6 Testing, Results and Evaluation **30**
6.1 Testing Approach 30
6.2 Pilot Deployment and Dataset 30
6.3 Measurement Methodology 31
6.4 Results 32

6.4.1 Position-Acquisition Latency 32
6.4.2 Source Resilience 32
6.4.3 ETA Estimation Behaviour 33
6.4.4 Passenger Engagement 33
6.5 The Idle-Reporting Problem and Optimization 33
6.5.1 The Discovery 33
6.5.2 The Two-Sided Fix 33
6.5.3 Verification 34
6.6 Requirements Coverage 34
6.7 Limitations 35

7 Conclusion and Future Work 36
7.1 Conclusion 36
7.2 Future Work 37
7.3 Closing Remark 37

References 39

A Tracker SMS Configuration Commands 41

B Operator-Configurable Processing Parameters 42

C API Surface: Functional Overview 43

D Passenger Application Screens 44

E Administration Console Screens 46

List of Figures

4.1 System architecture of UniBus Live 16
4.2 Real-time data flow 17
4.3 The trip lifecycle state machine. The PRE_TRIP state is what keeps the passenger's map informative before departure. 20
4.4 The 31-model relational schema, grouped into six functional domains (representative models shown for each domain). 21

5.1 Screens of the bilingual passenger application 26
5.2 Screens of the driver application 27
5.3 Principal screens of the administration console 28

D.1 Core tracking screens of the passenger application 44
D.2 Alerting and engagement screens 45

E.1 Management pages of the administration console 46
E.2 User management page 47

List of Tables

2.1 Positioning of UniBus Live against the reviewed literature. 9

3.1 Principal functional requirements. 11
3.2 Principal non-functional requirements. 12
3.3 Technology stack of UniBus Live. 14

6.1 Pilot deployment summary. 31
6.2 Position-acquisition latency (device fix to server receipt). 32
6.3 Idle-reporting efficiency optimization. 34
6.4 Requirements coverage demonstrated in the pilot. 35

A.1 GT06S SMS commands used in this work. 41

B.1 Pipeline parameters and pilot values. 42

C.1 Functional overview of the API surface. 43

List of Abbreviations

ACC	Accessory (vehicle ignition sense line)
API	Application Programming Interface
APK	Android Package
ETA	Estimated Time of Arrival
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
GSM	Global System for Mobile Communications
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IoT	Internet of Things
JSON	JavaScript Object Notation
JWT	JSON Web Token
MAE	Mean Absolute Error
ORM	Object–Relational Mapper
OTP	One-Time Password
REST	Representational State Transfer
SIM	Subscriber Identity Module
SMS	Short Message Service
SQL	Structured Query Language
TCP	Transmission Control Protocol
UI	User Interface
UX	User Experience
VPS	Virtual Private Server

CHAPTER 1

Introduction

1.1 Background

For a large share of the students of any university, the day begins and ends with a bus. University transport is often the only affordable way to travel between home and campus, and in Bangladesh—where many students commute considerable distances into district towns such as Manikganj—the university bus is a daily necessity rather than a convenience. Yet the information available to the rider about that bus has barely changed in decades. The timetable on a notice board says when the bus is *supposed* to leave; nothing tells a student where the bus *actually* is right now.

Meanwhile, real-time vehicle tracking itself is a mature technology. Satellite positioning receivers are inexpensive, cellular data coverage reaches nearly every road a university bus travels, and every student carries a smartphone capable of rendering a live map. Ride-sharing platforms have made the "watch your vehicle approach on the map" experience an everyday expectation. The technical ingredients for live bus tracking are therefore all available; what has been missing at institutions like ours is a system that assembles those ingredients at a cost a university transport office can justify, in a form students will actually use, and in the language they speak.

1.2 Motivation

The motivation for this work came directly from our own daily experience and that of our fellow students. A student who arrives at a roadside stop faces an information vacuum with real costs:

- **Missed buses.** If the bus passed early, the student may wait for a vehicle that has already gone, and discover it only when it is too late to arrange an alternative.
- **Wasted waiting time.** If the bus is late, the student stands at the stop—often without shelter, in heat or rain—for time that could have been spent at home or in the library had the delay been visible in advance.

- **Crowding and uncertainty at the origin.** Without knowing when a bus will actually depart, students cluster early around the boarding point, producing avoidable congestion.

Commercial fleet-tracking products could in principle address this, but three barriers make them a poor fit for a university in a developing region. First, their subscription pricing is set for commercial logistics fleets, not for a cost-sensitive academic transport office. Second, they are built for the fleet *operator*: their web dashboards answer "where are my vehicles?" for a manager, not "when will my bus reach my stop?" for a passenger. Third, their interfaces are almost always English-only, whereas a large portion of our student population is more comfortable in Bangla.

We therefore set out to build, deploy, and evaluate a complete system of our own—not a simulation or a prototype confined to a laboratory, but a platform running on the real university fleet with real riders.

1.3 Problem Statement

The problem addressed by this thesis can be stated as follows:

> *Students using university transport cannot observe the live position of their bus or a dependable estimate of its arrival time at their own stop. Existing commercial tracking solutions are too costly for institutional adoption, are designed for fleet operators rather than passengers, and are not localized. Furthermore, tracking approaches that depend on a driver's smartphone are fragile—they fail whenever the driver forgets, declines, or is unable to run the tracking application.*

Decomposing this, the specific technical problems to solve were:

1. **Autonomous position acquisition.** Obtain a continuous stream of bus positions without depending on any daily human action, while still tolerating the failure or absence of the tracking hardware.
2. **Noise robustness.** Convert raw, noisy GPS fixes—which contain urban scatter, occasional impossible jumps, and stationary jitter—into a stable position that can be shown to the public.
3. **Passenger-meaningful information.** Translate a raw position into what a rider actually needs: the arrival time at *their* stop, the state of the bus *before* departure, and an alert when their drop-off approaches.
4. **Low-latency delivery at low cost.** Push every update to every interested phone within seconds, using infrastructure cheap enough for a student-project budget.

5. **Localization.** Present the entire experience in both English and Bangla.

1.4 Objectives

The objectives of this thesis work were set at the start of the project and are listed here in the order in which the system must satisfy them:

1. To design a three-tier, event-driven architecture for real-time university bus tracking that runs entirely on a single low-cost server.
2. To implement resilient dual-source GPS acquisition, in which a fixed vehicle-grade GPS/GSM tracker acts as the fully automatic primary source and the driver's smartphone acts only as an optional automatic fallback, with per-fix health-based arbitration between them.
3. To implement a location-quality filtering pipeline that gates every fix on accuracy, physical plausibility, and minimum movement before it can move the publicly visible position.
4. To design a lightweight, training-data-free arrival-time estimator that produces a per-stop ETA with a confidence label on every accepted fix.
5. To build a bilingual (English/Bangla) passenger mobile application that remains informative in edge conditions—before departure, during signal loss, and near the rider's stop—and a web administration console for routes, schedules, fleet, and live operations.
6. To deploy the complete system on the operational university fleet and evaluate it over a multi-week pilot using real operational data.

1.5 Scope of the Work

The scope of this thesis covers the complete life of a position report: from the GPS receiver on the bus, through cellular transmission, decoding, arbitration, filtering, ETA computation, storage, and publication, to the pixel on a passenger's screen. It includes the hardware selection and configuration of the on-vehicle tracker, the backend server and database, the realtime delivery layer, the passenger application, the administration console, and the evaluation of the deployed whole.

Certain related problems are deliberately out of scope. The system does not attempt learning-based arrival prediction (it logs the data needed to train such models later); it does not process payments or ticketing; it does not measure passenger occupancy at scale; and the pilot was confined to one university's fleet in and around Manikganj rather than a multi-institution deployment. These boundaries are revisited as future work in

Chapter 7.

1.6 Contributions

The principal contributions of this thesis are:

1. **A deployed end-to-end architecture** for real-time university bus tracking centred on an autonomous fixed GPS tracker with an optional, health-arbitrated smartphone fallback, providing resilient coverage at low cost (Chapters 4 and 5).
2. **A lightweight geometric ETA estimator** that projects the live position onto the route polyline and derives per-stop arrival times from a rolling-average speed, annotated with a confidence level, and requiring no training data (Section 4.6).
3. **A passenger-facing design contribution**: a bilingual user experience with pre-trip vehicle awareness, staleness signalling, and proximity-based drop-off alerts, which keeps the interface informative precisely in the edge conditions where naive trackers show a blank map (Section 5.6).
4. **An operational evaluation** of the deployed system over a seven-week pilot on a real fleet, reporting acquisition latency, source-fallback usage, and passenger engagement (Chapter 6).
5. **The identification and elimination of idle-reporting waste**: pilot data showed that roughly 98% of raw device reports were transmitted while buses were parked; a two-sided optimization (device reporting cadence and server polling cadence) cut idle-period reporting by up to 60$\times$ without affecting live tracking (Section 6.5).

1.7 Organization of the Thesis

The remainder of this thesis is organized as follows. Chapter 2 reviews related work on campus bus tracking, smartphone-based positioning, arrival-time prediction, GPS quality handling, and open telematics platforms, and positions this work against it. Chapter 3 analyses the system's stakeholders and requirements and records the feasibility study and development methodology. Chapter 4 presents the system architecture and the design of each major component. Chapter 5 describes the implementation of the backend, the realtime layer, the two client applications, and the on-vehicle hardware configuration. Chapter 6 reports the seven-week pilot evaluation and discusses its findings, including the idle-reporting optimization. Chapter 7 concludes and outlines future work. The appendices record the tracker's SMS configuration commands, the operator-configurable parameters of the processing pipeline, and representative screens of the

passenger application.

CHAPTER 2

Literature Review

2.1 Overview

Real-time bus tracking sits at the intersection of several research areas: vehicle telematics, mobile and participatory sensing, arrival-time prediction, and GPS signal processing. This chapter reviews the work most relevant to a low-cost, passenger-facing university deployment, grouped into five strands: campus bus tracking systems (Section 2.2), smartphone-based and crowdsourced positioning (Section 2.3), bus arrival-time prediction (Section 2.4), GPS quality and map matching (Section 2.5), and open telematics platforms (Section 2.6). Section 2.7 then summarizes the gap that UniBus Live addresses.

2.2 Campus and Real-Time Bus Tracking Systems

Student-facing bus tracking has attracted steady attention because the need is so tangible. Sharif *et al.* [1] describe a real-time campus bus tracking mobile application in which an on-board GPS tracker is paired with an IoT passenger counter, so that riders see both the location of the bus and an indication of how full it is. Premkumar *et al.* [2] present a college bus tracking and notification system that combines GPS tracking with a schedule view and push notifications, so that students are actively told about the bus rather than having to watch a map continuously.

These systems demonstrate the practical value of live tracking on a campus fleet, and several of their ideas—push notifications, schedule integration—also appear in UniBus Live. However, they share two characteristics that limit their applicability to deployments like ours. First, each depends on a *single* on-board positioning source; if that device fails, is powered off, or is not yet installed on a particular vehicle, tracking for that bus simply stops. Second, their interfaces are English-only, which is a real adoption barrier for a student population that is more comfortable in Bangla. UniBus Live differs on precisely these two axes: it ingests two independent position sources with automatic

arbitration between them, and its entire passenger experience is bilingual.

2.3 Smartphone-Based and Crowdsourced Positioning

A complementary research line removes dedicated hardware entirely and uses smartphones as the position source. The seminal work of Zhou *et al.* [3] showed that bus arrival times can be predicted from passengers' phones alone: participatory sensing infers which passengers are on a bus and where that bus is, without any cooperation from the transit operator. Wepulanon *et al.* [4] develop this idea into a real-time arrival information framework driven entirely by crowdsourced smartphone observations, evaluated through simulation experiments.

The appeal of these approaches is obvious—zero hardware cost—but they carry a structural weakness for a small university fleet: coverage depends on enough contributing phones being on the bus at the right time. A route's first trip of the morning, a lightly loaded off-peak trip, or a trip whose riders decline to share location data all produce gaps precisely when a waiting passenger most needs information.

UniBus Live deliberately *inverts* the relationship between phones and hardware. Normal operation depends on no phone at all: the primary source is an autonomous, vehicle-fixed GPS tracker. A phone—the driver's, not the passengers'—participates only as an optional fallback that is selected automatically when no healthy hardware fix is available. Phone sensing thus supplements the system instead of carrying it.

2.4 Bus Arrival-Time Prediction

Predicting when a bus will arrive is a well-studied problem with a spectrum of approaches. Reich *et al.* [5] survey ETA prediction methods for public transport and organize them by the input data they require, while Singh and Kumar [6] review artificial-intelligence approaches specifically for bus arrival-time prediction.

On the data-driven end of the spectrum, Ye and Thiengburanathum [7] demonstrate support-vector regression running on a Spark big-data framework for city buses in Chiang Mai; Rashvand *et al.* [8], [9] train fully connected neural networks that predict arrival and departure times across hundreds of routes; Jeong *et al.* [10] apply a transformer encoder to capture long-range temporal dependence in bus trajectories; and Google's BusTr system, described by Barnes *et al.* [11], translates real-time traffic forecasts into bus delay predictions at a global scale. These models achieve strong accuracy, but they share a prerequisite that a brand-new single-campus deployment cannot meet: large historical trajectory datasets, and in some cases external traffic feeds, must exist before the

model can be trained at all.

UniBus Live therefore takes the pragmatic position appropriate to a cold start: a lightweight *geometric* estimator (remaining along-route distance divided by a rolling-average speed) that needs no training data, paired with systematic logging of every prediction so that a data-driven model can be trained later, once sufficient operational history has accumulated. The estimator is described in Section 4.6.

2.5 GPS Quality, Filtering, and Map Matching

Raw GPS is noisy, and a public-facing tracker cannot simply display every fix it receives. The classical treatment of reconciling noisy fixes with a road network is the Hidden Markov Model map-matching method of Newson and Krumm [12], which jointly considers measurement noise and road-network topology to recover the most probable path. HMM map matching is the right tool when the vehicle could be anywhere on an arbitrary road network.

A university bus, however, is not anywhere: it follows a small set of fixed, pre-defined routes. UniBus Live exploits this constraint to use a far lighter pipeline—an accuracy gate, a physical-plausibility (teleport) guard, a stationary-jitter floor, and projection of each fix onto the known route polyline. This subset is sufficient to stabilize the displayed position and to drive along-route ETA computation, at a small fraction of the computational cost of full map matching (Section 4.5).

2.6 Open Telematics Platforms

Commercial GPS trackers speak a large variety of vendor-specific binary protocols. Open-source telematics servers such as Traccar [13] exist precisely to absorb this diversity: Traccar implements protocol decoders for a wide range of tracker models and exposes the decoded positions through a REST API and WebSocket stream, along with geofencing primitives. Building on such a platform means a project does not reimplement low-level device communication.

UniBus Live adopts this approach: a self-hosted Traccar instance handles raw device ingestion for the Concox GT06S tracker, and the entire passenger-facing, ETA-aware, bilingual application layer is built on top of it as an independent application server (Section 4.3).

2.7 Research Gap and Positioning of This Work

Table 2.1 summarizes the review. Across the reviewed strands, three gaps recur. First, campus tracking prototypes rely on a single position source and stop working when it fails. Second, crowdsourced systems avoid hardware but inherit participation-dependent coverage. Third, accurate ETA models presuppose historical data that a new deployment does not have, and most reported systems are either simulation-based or English-only.

Table 2.1: Positioning of UniBus Live against the reviewed literature.

Strand	Typical limitation	UniBus Live response
Campus trackers [1], [2]	Single source; English-only	Dual-source with arbitration; bilingual UX
Crowdsourced sensing [3], [4]	Coverage needs participants	Autonomous fixed tracker; phone only as fallback
Data-driven ETA [7]–[11]	Needs large training history	Training-free geometric ETA; logs data for future models
HMM map matching [12]	Heavier than needed for fixed routes	Route-constrained filtering and projection
Telematics platforms [13]	Operator-facing, no passenger layer	Passenger-centric application layer on top

In contrast to single-source prototypes, simulation-only studies, and data-hungry prediction models, UniBus Live contributes a *deployed*, low-cost system that combines source resilience, a training-data-free ETA estimator, and a localized, edge-condition-aware passenger experience, evaluated on a real campus fleet over seven weeks. To the best of our knowledge, no previously reported university bus tracking system combines these properties.

2.8 Summary

This chapter surveyed five strands of related work and identified the recurring gaps: single-source fragility, participation-dependent coverage, training-data prerequisites, and the absence of localized, passenger-centric deployments. The next chapter translates the resulting opportunity into concrete requirements for UniBus Live.

CHAPTER 3

System Analysis and Requirements

3.1 Stakeholder Analysis

Four groups of stakeholders interact with the system, each with distinct needs:

Passengers (students and staff). The primary beneficiaries. They need to see the live position of their bus, the estimated arrival time at *their own* stop, the day's schedules, and timely alerts—in English or Bangla according to their preference. They interact only with the mobile application.

Drivers. Deliberately, drivers have *no required role* in normal operation: the fixed tracker reports automatically. A driver participates only in the optional fallback path, by carrying a smartphone that publishes location when a bus has no working tracker.

Transport administrators. They manage routes, stops, schedules, buses, GPS devices, and driver assignments, and monitor the whole fleet on a live operations map. They interact with the web administration console.

The institution. The university requires the recurring cost to stay within a modest budget, and the deployment to run on its real fleet without disrupting operations.

3.2 Functional Requirements

The functional requirements were derived from the stakeholder analysis and refined during early development. Table 3.1 lists the principal requirements; each is referenced from the design chapter where it is satisfied.

Table 3.1: Principal functional requirements.

ID	Area	Requirement
FR-1	Acquisition	Ingest positions automatically from a fixed vehicle GPS tracker with no driver involvement.
FR-2	Acquisition	Accept authenticated position reports from a driver smartphone as an optional secondary source.
FR-3	Acquisition	Select the best available source per fix, automatically and without passenger-visible artefacts.
FR-4	Quality	Reject inaccurate, physically implausible, and jittery fixes before they affect the public position.
FR-5	ETA	Compute an arrival estimate for every upcoming stop of the active route on each accepted fix, with a confidence label.
FR-6	Delivery	Push position and ETA updates to all subscribed passengers within seconds over a persistent connection.
FR-7	Lifecycle	Show the bus state before departure (parked, approaching origin, arrived at origin) instead of a blank map.
FR-8	Lifecycle	Mark displayed data as stale when the latest fix ages beyond a threshold.
FR-9	Alerts	Notify a subscribed rider when the bus reaches an 80 m radius of a stop, including their chosen drop-off stop.
FR-10	Localization	Provide the complete passenger experience in both English and Bangla, switchable in-app.
FR-11	Administration	Manage routes, stops, schedules, buses, GPS devices, and user roles through a web console.
FR-12	Administration	Display all active vehicles on a live operations map for transport staff.
FR-13	Audit	Log every raw report, every accepted fix, and every ETA prediction for later analysis.

3.3 Non-Functional Requirements

Table 3.2: Principal non-functional requirements.

ID	Quality	Requirement
NFR-1	Latency	A position fix should reach subscribed passengers within seconds of capture under normal network conditions.
NFR-2	Resilience	Loss of any single position source must degrade coverage gracefully, not eliminate it.
NFR-3	Cost	The entire backend must run on one low-cost virtual private server; no managed cloud services are required.
NFR-4	Efficiency	Reporting and polling cadence must not waste cellular data or vehicle battery while buses are parked.
NFR-5	Security	All client–server communication is authenticated; role-based authorization separates passengers from administrators.
NFR-6	Scalability	Realtime fan-out must be able to scale horizontally across server processes without losing consistency.
NFR-7	Usability	The passenger app must remain informative during edge conditions (pre-trip, signal loss, stationary bus).

NFR-4 deserves a remark: it was *added during the pilot*, when operational data revealed that roughly 98% of raw device reports were being transmitted while buses were parked. The discovery and the resulting optimization are treated fully in Section 6.5; the requirement is recorded here because it reshaped both the device configuration and the server's polling design.

3.4 Feasibility Study

3.4.1 Technical Feasibility

Every component of the proposed system relies on proven, freely available technology: a mass-produced vehicle GPS/GSM tracker (Concox GT06S), an open-source telematics server (Traccar) that already implements the tracker's binary protocol, an open-source relational database (PostgreSQL), an in-memory store (Redis), and mature application frameworks (Node.js/Express for the server, React Native/Expo for the mobile app, Next.js for the console). No component required custom hardware or proprietary licensing, so technical feasibility was established by construction.

3.4.2 Economic Feasibility

The recurring infrastructure is a single low-cost virtual private server hosting the application server, Traccar, PostgreSQL, and Redis together. The one-time hardware cost is the tracker unit and its SIM per bus. Both figures sit comfortably within a student-project budget, and—more importantly for adoption—within what a university transport office could sustain for a full fleet. The economic design goal, "everything on one server, no external paid services," held throughout the pilot.

3.4.3 Operational Feasibility

Operational feasibility hinged on one question: can the system run without adding work for anyone? The hardware-first design answers it. The tracker is hard-wired to the bus and reports automatically; drivers need not carry, charge, or operate anything; administrators interact with the console only to maintain routes and schedules, which they already did on paper. The seven-week unattended pilot (Chapter 6) confirmed this in practice.

3.5 Development Methodology

The system was developed iteratively. An initial vertical slice—one bus, one route, raw positions on a map—was deployed early, and each subsequent iteration added one capability (quality filtering, ETAs, trip lifecycle, notifications, bilingual UI, the administration console) followed by observation on the live fleet. Two practices shaped the result. First, *operational data drove priorities*: the staleness indicator, the pre-trip map state, and ultimately the idle-reporting optimization all originated from watching real deployment behaviour rather than from the original plan. Second, *every layer logs*: raw ingests, accepted fixes, trip events, and ETA predictions are all persisted, which is what later made the evaluation in Chapter 6 possible directly from the production database.

3.6 Tools and Technologies

Table 3.3 summarizes the technology choices; the rationale for each appears in the corresponding design section. All server and client components are built on freely available, industry-standard platforms: the Node.js runtime [14] with the Express framework [15] and the Prisma ORM [16] on the server; PostgreSQL [17] and Redis [18] for persistence and messaging; Socket.IO [19] for realtime delivery; and React Native [20] with Expo [21] for the mobile applications, with Next.js [22] for the web console.

Table 3.3: Technology stack of UniBus Live.

Layer	Technology	Role
Vehicle	Concox GT06S	Fixed GPS/GSM tracker (primary source)
Vehicle	Android smartphone	Optional driver-side fallback source
Ingestion	Traccar	Tracker protocol decoding, device management
Server	Node.js + Express (TypeScript)	Application server
ORM	Prisma	Typed access to the relational schema
Database	PostgreSQL	System of record (31 models)
Cache/bus	Redis	Hot-path cache and pub/sub back-plane
Realtime	Socket.IO	WebSocket rooms per active trip
Mobile	React Native + Expo	Bilingual passenger application
Mobile	React Native + Expo	Driver app (fallback publisher)
Web	Next.js (React)	Administration console
Maps	Google Maps APIs	Map rendering and route building
Hosting	Single VPS	All backend components self-hosted

3.7 Summary

This chapter identified the system's four stakeholder groups, fixed thirteen functional and seven non-functional requirements, established technical, economic, and operational feasibility, and recorded the iterative, data-driven development methodology and technology stack. The next chapter presents the design that satisfies these requirements.

CHAPTER 4

System Design

4.1 Architectural Overview

UniBus Live follows a three-tier, event-driven architecture with four cooperating components: a passenger mobile application, a web-based administration console, a central application server, and a vehicle GPS-acquisition layer. Two communication styles are used side by side, each where it fits: stateless REST over HTTPS for command and query operations (log in, fetch schedules, manage routes), and a persistent WebSocket channel for the one thing that genuinely needs push semantics —the live vehicle state. Figure 4.1 shows the complete architecture.

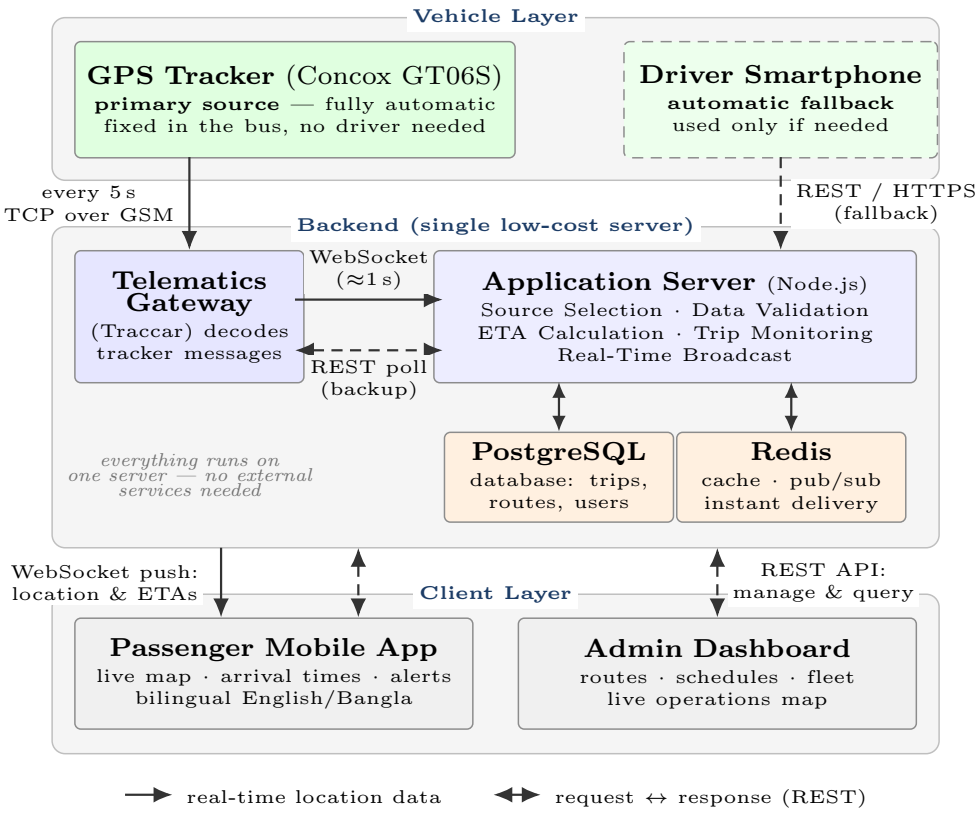


Figure 4.1: System architecture of UniBus Live. The GPS tracker is the primary, fully automatic source; the driver's smartphone is only a fallback. All backend components share one low-cost server. Solid arrows carry real-time location data; dashed double-headed arrows are REST request/response paths.

Three design decisions define the architecture:

1. **Hardware-first acquisition.** The primary position source is a fixed, vehicle-grade tracker that operates with no human involvement. Anything a person must remember to do daily will eventually not be done; the design removes that failure mode entirely, and demotes the smartphone to an optional fallback (Section 4.3).
2. **A single low-cost server.** The application server, telematics gateway, database, and cache are co-hosted on one virtual private server. This satisfies NFR-3 and simplifies operations, while the Redis pub/sub back-plane preserves the option of scaling the realtime tier horizontally later (Section 4.9).
3. **Event-driven processing.** Each arriving fix triggers a processing chain—arbitrate, filter, estimate, persist, publish—and everything downstream of the fix is pushed, never polled, so passenger latency is bounded by acquisition rather than by client refresh cycles.

4.2 Real-Time Data Flow

Figure 4.2 traces the life of a single position report through the system, grouped into three phases: acquisition (capture and transmission), processing (arbitration, filtering, ETA computation), and delivery (persistence, publication, and push). The seven steps of the figure correspond to the design sections of this chapter and the implementation sections of Chapter 5.

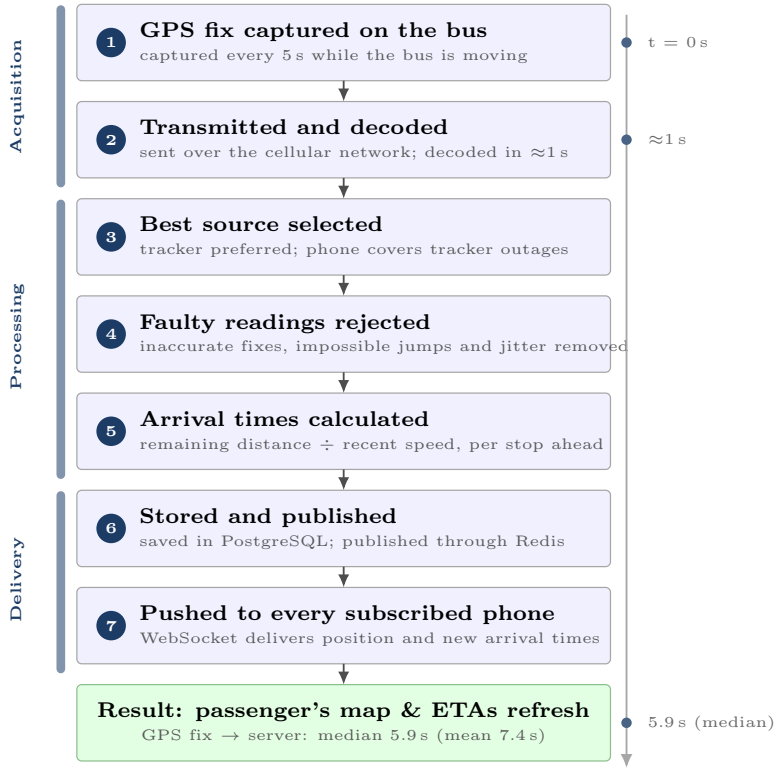


Figure 4.2: Real-time data flow: the life of one position report, from the GPS fix on the bus to the passenger's screen. The timeline on the right shows representative latencies measured in the pilot; the median end-to-end acquisition latency was 5.9 s.

4.3 GPS-Acquisition Layer

4.3.1 Primary Source: Fixed Vehicle Tracker

The primary source is a Concox GT06S, a vehicle-grade GPS/GSM tracker hard-wired to the bus's electrical system; the satellite-positioning principles underlying such receivers are treated comprehensively by Kaplan and Hegarty [23]. While the vehicle moves, the unit obtains a satellite fix every 5 seconds and transmits it over the cellular network (TCP over GSM) to a self-hosted Traccar telematics server, which implements the tracker's binary protocol and decodes each message within about a second. Because the tracker is fixed to the vehicle and powered from it, tracking requires no driver ac-

tion, no driver-owned device, and no driver data plan—the property on which the whole operational-feasibility argument rests.

4.3.2 Secondary Source: Driver Smartphone (Optional)

A bus without a working tracker—not yet fitted, faulty, or SIM depleted—can still be tracked. A driver smartphone running the app in driver mode publishes its location to an authenticated REST endpoint. This path is deliberately *optional*: it exists so that coverage degrades gracefully, not because the system depends on it. During the pilot, two of the three buses were not yet fitted with trackers, so this fallback path carried a substantial share of real trips (Section 6.4.2).

4.3.3 Ingesting from the Telematics Gateway

The application server consumes Traccar's decoded positions through two complementary paths, designed so that the faster one is used whenever possible and the slower one guarantees delivery:

- **Push (preferred).** A session-cookie WebSocket subscription to Traccar delivers each new fix within about one second of decoding.
- **Poll (backup).** A REST polling job queries Traccar every 8 seconds during active trips, covering intervals when the socket is unavailable. (After the idle-efficiency optimization of Section 6.5, polling drops to every 60 seconds for buses with no active or imminent trip.)

4.4 Source Arbitration Design

When both a tracker and a driver phone are reporting for the same bus, exactly one of them must drive the position passengers see. The design maintains, per bus, one *source-tracking state* for each active source and a single *canonical tracking state* for the public position. On every inbound fix, an arbitration routine scores each source's health—essentially its recency and validity—and selects the canonical source by health and a configurable priority rank that prefers the hardware tracker. Because the decision is re-evaluated per fix, a bus can hand over from tracker to phone (or back) in the middle of a trip with no passenger-visible artefact: the canonical marker simply continues from the freshest healthy source. This satisfies FR-3 and NFR-2.

4.5 Location-Quality Filtering Design

Raw GPS cannot be displayed as-is. The design places a filtering pipeline between arbitration and everything downstream, so that only credible fixes can move the canonical position. Four operator-configurable rules gate each fix:

1. **Accuracy gate.** Fixes reporting accuracy worse than 120 m are soft-rejected; worse than 250 m, hard-rejected.
2. **Teleport guard.** A fix implying an inter-fix speed above 120 km/h is physically implausible on the pilot routes and is rejected.
3. **Minimum-movement floor.** Displacements under 4 m are treated as stationary, suppressing the GPS jitter that would otherwise make a parked bus wander on the map.
4. **Freeze/release hysteresis.** Under sustained poor accuracy the marker is held still, and motion resumes only on a confident displacement, preventing oscillation at the accuracy boundary.

The thresholds are deliberately conservative: 120 km/h exceeds any plausible bus speed on the pilot routes, and the 4 m floor sits above the tracker's typical urban scatter, so legitimate motion is never suppressed. A short-window rolling average of accepted speeds is maintained per trip and feeds the ETA estimator. Full HMM map matching (Section 2.5) was considered and rejected: the buses follow a small set of fixed routes, so projecting onto the known polyline achieves the needed stability at a fraction of the cost.

4.6 Arrival-Time Estimation Design

The ETA estimator is geometric and training-free, chosen so the system is fully functional from its first day of deployment (Section 2.4 explains why data-driven models were not an option at cold start).

The route is first reduced to a cumulative along-path distance array over its ordered stops. Each accepted fix is projected onto the nearest route segment using a planar (local equirectangular) projection, which yields the bus's *progress* distance along the route and its cross-track offset; the projection makes the estimate robust to lateral GPS error and to the bus running slightly off the idealized polyline. For every stop still ahead, the remaining distance is the difference between the stop's cumulative distance and the bus's progress; dividing by an effective speed yields that stop's ETA. The effective speed is the per-trip rolling average when it is reliable (≥ 5 km/h), otherwise a configurable default of 20 km/h. Each estimate carries a HIGH/MEDIUM/LOW confidence

label derived from the quality of the speed signal, so the interface can be honest about uncertainty. A stop is marked *reached* when the bus enters an 80 m arrival radius, which triggers a stop-arrival event and, for subscribed riders, a push notification (FR-9). The complete procedure is given as Algorithm 1.

Algorithm 1 Estimating the ETA to every upcoming stop

Require: current bus GPS fix; ordered route stops; recent average speed
Ensure: an ETA and a confidence label for each stop ahead

Compute each stop's distance from the route start (cumulative)
Project the bus fix onto the nearest route segment to find how far along the route it has travelled (its *progress*)
if recent average speed ≥ 5 km/h **then**
    $speed \leftarrow$ recent average speed
else
    $speed \leftarrow 20$ km/h ▷ fallback default
end if
for all stops ahead of the bus **do**
    $remaining \leftarrow$ (stop's distance) $-$ (progress)
    ETA $\leftarrow remaining / speed$
end for
Assign a confidence (High/Medium/Low) from the speed quality
if the nearest stop is within 80 m **then**
    mark it reached and notify subscribed riders
end if
return the ETA and confidence for every stop ahead

4.7 Trip Lifecycle Design

A trip is modelled as a state machine, shown in Figure 4.3: PLANNED → PRE_TRIP → RUNNING → ENDED.

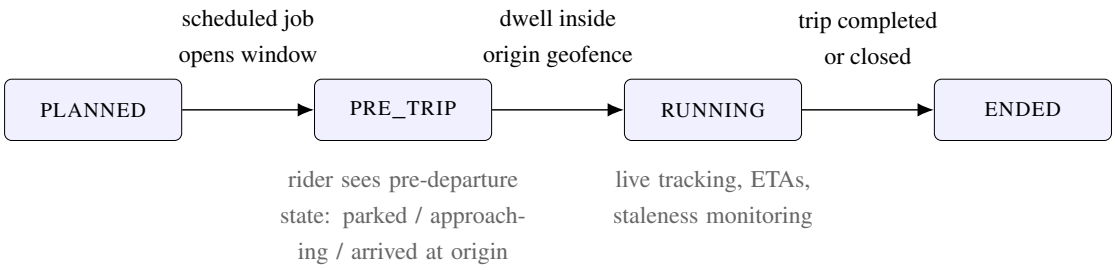


Figure 4.3: The trip lifecycle state machine. The PRE_TRIP state is what keeps the passenger's map informative before departure.

The distinctive state is PRE_TRIP: a scheduled job opens a window before the scheduled departure during which the rider sees the bus's pre-departure state —parked, approaching the origin, or arrived at the origin—instead of a blank map (FR-7). The vehicle auto-promotes to RUNNING when it dwells inside the origin geofence. A staleness job runs every 30 seconds and flags any trip whose latest fix has aged beyond a threshold, so a frozen marker is presented as *stale* rather than silently passed off as live (FR-8). Honesty about data age is treated as a first-class interface requirement, not an afterthought.

4.8 Database Design

PostgreSQL is the system of record, accessed through the Prisma ORM. The schema comprises 31 relational models organized into six functional domains, shown in Figure 4.4.

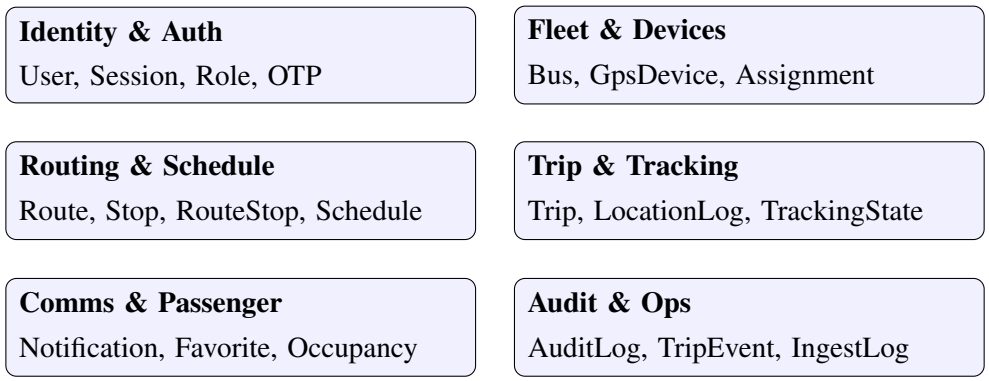


Figure 4.4: The 31-model relational schema, grouped into six functional domains (representative models shown for each domain).

Two domains matter most to the thesis argument. *Trip & Tracking* separates the per-source tracking states from the canonical tracking state, which is what makes the arbitration design of Section 4.4 expressible in data. *Audit & Ops* preserves every raw ingest and trip event; this is the layer that made the retrospective analysis of Chapter 6— including the discovery of the 98% idle-reporting waste—possible without any additional instrumentation.

4.9 Realtime Delivery Design

Realtime delivery uses Socket.IO over WebSocket, the IETF-standardized protocol that provides a persistent, full-duplex channel over a single TCP connection [24]. Each active trip is a logical *room*; a passenger tracking a trip joins that trip's room and thereafter receives only that trip's events, bounding per-client traffic to the single vehicle being observed. Redis serves as the publish/subscribe back-plane: accepted canonical updates

and recomputed ETAs are published once, and every server process holding subscribers for that trip forwards them. This lets the realtime tier scale horizontally without losing fan-out consistency (NFR-6), even though the pilot ran comfortably on a single process. Authentication is enforced both at socket connection and at room join (NFR-5).

Offline behaviour is part of the same design: the client retains the most recent state, so a transient disconnection degrades to a "last known" view with an explicit staleness indicator rather than an empty screen.

4.10 Security Design

All client–server traffic runs over HTTPS. Passengers and administrators authenticate against the identity domain of the schema; role-based authorization separates passenger, driver, and administrator capabilities. The driver-side location endpoint accepts reports only from an authenticated driver bound to the bus in question, preventing arbitrary clients from injecting positions. Socket connections carry the same identity, and room joins are authorized per trip. Administrative actions are recorded in the audit domain.

4.11 Summary

This chapter presented the three-tier, event-driven architecture of UniBus Live and the design of each major component: dual-source GPS acquisition with per-fix health arbitration, the four-gate location-quality filter, the training-free geometric per-stop ETA estimator, the trip lifecycle with its pre-trip and staleness states, the 31-model relational schema, the room-based realtime delivery layer, and the security model. Chapter 5 describes how each design was realized.

CHAPTER 5

Implementation

5.1 Backend Application Server

The application server is written in TypeScript and runs on Node.js with the Express framework. TypeScript was chosen over plain JavaScript because the domain model is intricate—trips, sources, tracking states, and their transitions—and static typing catches an entire class of mistakes at compile time that would otherwise surface as runtime faults on a live fleet.

Persistence goes through the Prisma ORM to PostgreSQL. Prisma generates typed client code from the declarative schema, so a query that survives compilation is structurally valid against the database. The 31 models of the schema (Section 4.8) map one-to-one to Prisma model declarations, and schema migrations are versioned alongside the application code.

The server exposes two interfaces: a REST API (authentication, schedules, routes, favourites, administration) and the Socket.IO realtime gateway. Internally, the position-processing chain is implemented as a sequence of services—ingestion, arbitration, filtering, ETA, publication—each of which appends to the audit logs as it acts (FR-13).

5.2 Traccar Integration

A self-hosted Traccar instance decodes the Concox GT06S binary protocol. The application server consumes it through the two paths designed in Section 4.3:

- The **WebSocket push** path authenticates to Traccar with a session cookie and subscribes to its live position stream; a decoded fix reaches the application server within about one second.
- The **REST poll** path runs on a timer and reconciles the latest positions for all devices, guaranteeing progress when the socket is down. Its cadence is trip-aware: 8 s for buses in an active trip or pre-trip window, 60 s otherwise (the optimization

of Section 6.5).

Every inbound fix—pushed, polled, or posted by a driver device—is recorded with both its device-reported timestamp and its server-receipt timestamp and tagged with its source. The difference between those two timestamps is precisely the position-acquisition latency reported in Chapter 6, so the evaluation methodology was built into the ingestion path from the start.

5.3 Source Arbitration and Filtering

The arbitration routine of Section 4.4 runs on every inbound fix. Health scoring is intentionally simple—recency and validity dominate—because the failure mode it must detect is blunt: a source going silent. The configurable priority rank prefers the hardware tracker whenever it is healthy. During the pilot, mid-trip handovers occurred and produced no passenger-visible discontinuity; the canonical marker continued from the freshest healthy source, which is the exact behaviour the design called for.

The filtering pipeline implements the four gates of Section 4.5 with the thresholds listed in Appendix B (accuracy soft/hard limits 120 m/250 m, teleport guard 120 km/h, movement floor 4 m). Accepted fixes update the canonical state, append to the per-trip location log, and refresh the rolling speed window; rejected fixes are logged with their rejection reason, which proved invaluable when tuning the thresholds early in the pilot.

5.4 ETA Computation

The estimator implements Algorithm 1 directly. Route geometry is preprocessed once into a cumulative-distance array; per fix, the projection and the per-stop loop are a few hundred floating-point operations, so the estimator adds no measurable load even at full reporting rate. Every prediction is logged with its trip, stop, predicted instant, and confidence label, building the dataset against which mean-absolute-error can be measured as arrivals accumulate—and from which a learned predictor can later be trained (Section 7.2).

5.5 Trip Lifecycle and Background Jobs

Three scheduled jobs drive the lifecycle of Section 4.7:

- The **pre-trip job** opens the PRE_TRIP window ahead of each scheduled departure, computes the vehicle's pre-departure state (parked, approaching origin, arrived at

origin), and promotes the trip to RUNNING when the bus dwells inside the origin geofence.

- The **staleness job** runs every 30 s and flags trips whose latest fix has aged beyond the threshold, switching the passenger view to its explicit stale presentation.
- The **polling job** implements the trip-aware Traccar reconciliation cadence described above.

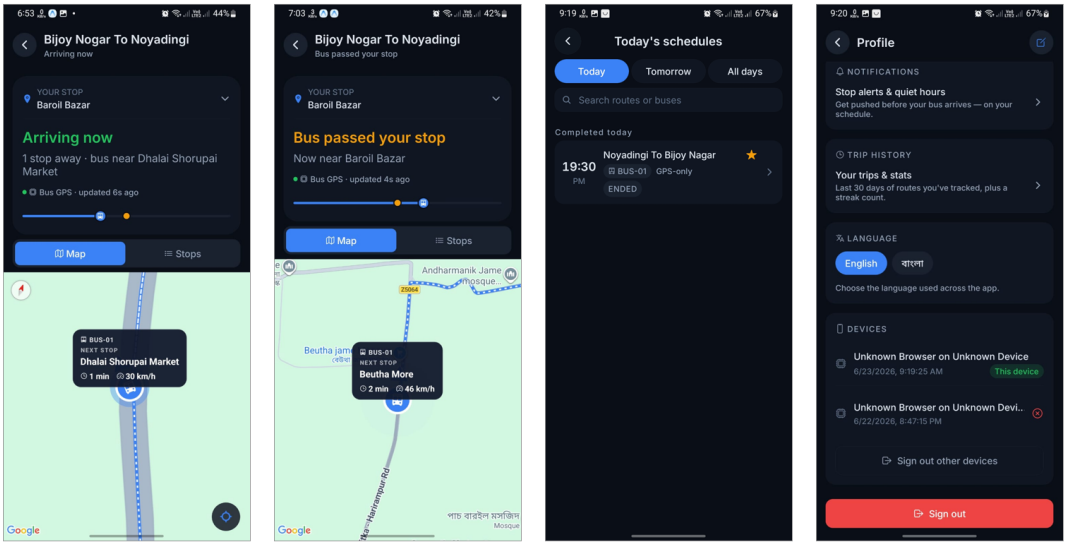
5.6 Passenger Mobile Application

The passenger application is built with React Native and Expo, targeting Android first (the platform of essentially all pilot riders). Its central screen is a live map showing the route polyline, its stops, and the bus marker with a callout presenting the vehicle's label, next stop, ETA, and current speed.

Four behaviours distinguish it from a naive tracker, each an edge condition handled by design:

- **Pre-trip awareness.** During the PRE_TRIP window the app renders the bus at its known pre-departure position (typically the depot or origin), so the map is never blank while a rider waits for departure (FR-7).
- **Staleness signalling.** When the server flags a trip stale, the app says so explicitly instead of letting a frozen marker masquerade as live data (FR-8).
- **Offline retention.** On a transient disconnection the app keeps the last known state visible, marked as such, and resumes seamlessly on reconnect.
- **Drop-off alerts.** A rider selects their stop; when the bus enters the 80 m arrival radius, a push notification fires even if the app is backgrounded (FR-9), using a foreground location service for continued tracking.

The interface is fully bilingual. Every string in the application exists in English and Bangla, switchable in-app at any time (FR-10), and place names render in both scripts. Localization was implemented as a first-class concern from the first screen onward—retrofitting translation onto a finished English app tends to leave exactly the gaps that erode user trust. Representative screens are shown in Figure 5.1 (Appendix D reproduces them at full page width).



(a) Live tracking with ETA (b) On-route: next stop & speed (c) Today's schedules (d) Language: English / Bangla

Figure 5.1: The bilingual passenger application. (a) live tracking of a moving bus with the next-stop ETA; (b) on-route status showing the next stop and current speed; (c) today's schedules; (d) the in-app English/Bangla language toggle. Place names render in both scripts.

5.7 Driver Mobile Application (Fallback Publisher)

The optional smartphone fallback of Section 4.3 is served by a separate, purpose-built driver application (React Native and Expo, distributed directly to drivers rather than publicly). It is deliberately minimal—a login screen and a single home screen—because its only job is to publish the bus position when a vehicle has no working tracker. Figure 5.2 shows its principal states.

Four design decisions define it:

- **Explicit sharing lifecycle.** The app shares nothing until the driver taps *Start trip*, and stops the moment the trip ends; between trips it shows an explicit "not sharing" state. Location is therefore published only during an active trip.
- **Background publishing.** Unlike the passenger app, the driver app declares Android background-location permissions and runs a foreground service, with the publishing task registered at the global JavaScript scope so the operating system can invoke it even when the screen is off. Each fix is posted to the authenticated driver-location REST endpoint (FR-2), reading the active trip and credentials from secure storage.
- **Source selection in view.** The home screen carries the *bus device / my phone* source selector and, while live, shows the driver the same telemetry the server sees—current speed, GPS accuracy, and seconds since the last update—so a

driver can confirm at a glance that publishing is working.

- **Role guarding.** Only accounts with the driver role can proceed past login; any other account is refused inside the app.

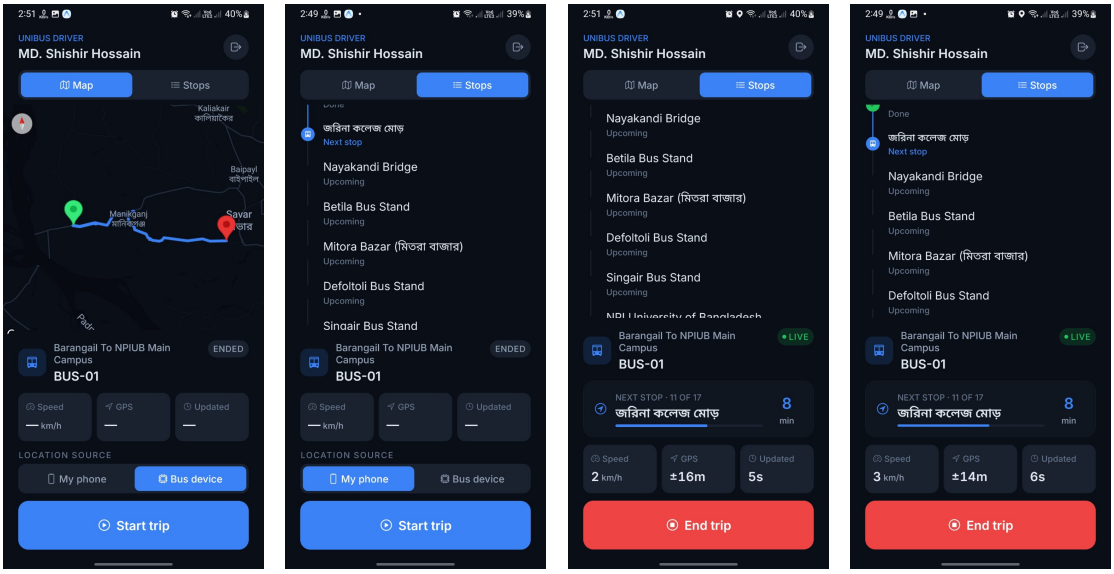


Figure 5.2: The driver application. Left to right: (a) the assigned trip's route map with the location-source selector (bus device / my phone) and the *Start trip* control; (b) trip preparation with *my phone* chosen as the location source and the upcoming stop sequence listed; (c) an active trip publishing LIVE: next stop (11 of 17) with its ETA, current speed, GPS accuracy, seconds since the last update, and the *End trip* control; (d) the stop-by-stop progress view with done/next/upcoming states, stop names rendered in both scripts.

5.8 Administration Console

The administration console is a server-rendered React application built with Next.js. It integrates the Google Maps JavaScript API for two map-centric tools: the *live operations map*, which shows every active vehicle in the fleet at once (FR-12), and the *route builder*, with which an administrator lays out a route's polyline and stop sequence directly on the map. Figure 5.3 shows four of its principal screens; Appendix E reproduces the remaining management pages. Beyond the maps, the console manages buses, GPS devices, schedules, and user roles through the same REST API the mobile app uses (FR-11)—one API surface, two clients, no duplicated logic.

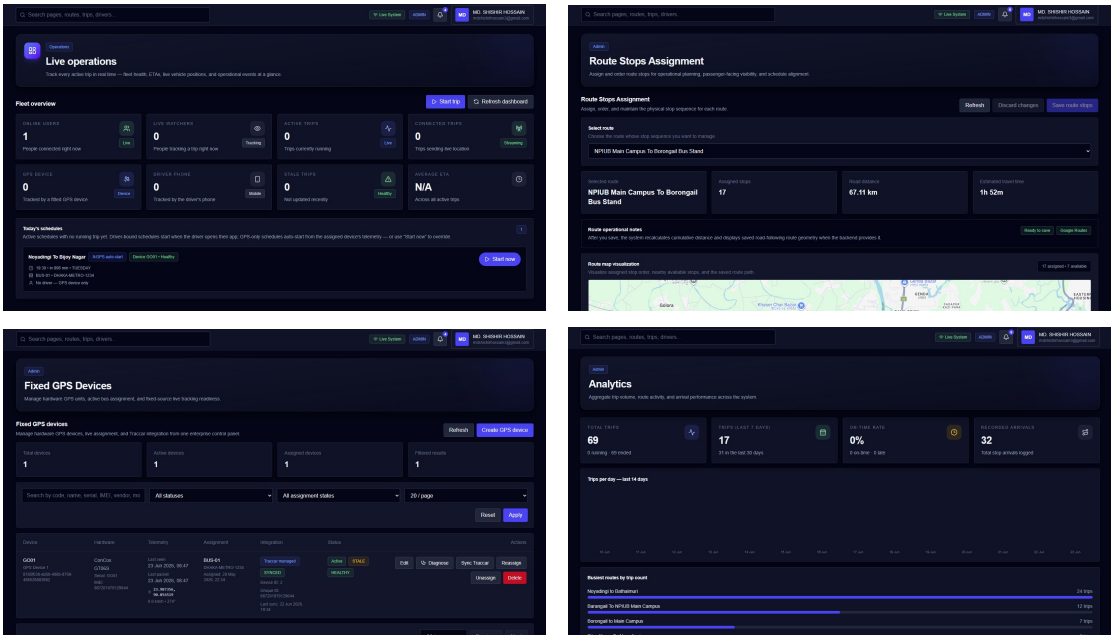


Figure 5.3: Principal screens of the administration console. Top left: the live operations dashboard with the fleet overview and today's schedules. Top right: the route-builder view, assigning the ordered stop sequence of a route with its along-path distance, estimated travel time, and map visualization. Bottom left: the fixed GPS device registry with telematics-integration status and per-device diagnostics. Bottom right: the analytics view aggregating trip volume, route activity, and arrival performance.

5.9 On-Vehicle Device Configuration

The Concox GT06S is configured over SMS, and its configuration became an integral part of the system's efficiency story. The unit as initially deployed transmitted a fix every 5 seconds *regardless of vehicle state*—including all night, parked in the depot. After the pilot data quantified this waste (Section 6.5), we set an asymmetric reporting cadence directly on the device:

- `TIMER,5,300#` — report every 5 s while moving, but only every 300 s while static;
- `PARAM#` — read back the active configuration for verification;
- `STATUS#` — report battery and GSM/GPS state;

each acknowledged by return SMS (the full command set appears in Appendix A). We verified the change from two independent vantage points—the per-position timestamps in Traccar's report view and the server's own ingest log—confirming parked-state fixes spaced by minutes rather than seconds, while a live test drive still produced dense five-second fixes. A low-frequency heartbeat keeps the device visible online between reports. Wiring the ignition (ACC) line for engine-state-triggered sleep is identified as

future work.

5.10 Deployment

The entire backend—application server, Traccar, PostgreSQL, and Redis—runs on a single low-cost virtual private server, satisfying NFR-3. The passenger application is distributed as an Android APK built through the Expo Application Services build pipeline; the administration console is served from the same VPS. No managed cloud services, message queues, or third-party realtime providers are involved: the design goal that a university could replicate this deployment for the cost of one small server and one tracker per bus held from the first day of the pilot to the last.

5.11 Summary

This chapter described the realization of the design: the TypeScript application server and its typed persistence layer, the dual push/poll Traccar integration with per-fix latency instrumentation, the arbitration and filtering services, the direct implementation of the ETA algorithm, the three lifecycle jobs, the bilingual passenger application with its four edge-condition behaviours, the dedicated driver application that implements the fallback publishing path, the administration console, the SMS-based tracker configuration, and the single-server deployment. The next chapter evaluates the deployed whole on the real fleet.

CHAPTER 6

Testing, Results and Evaluation

6.1 Testing Approach

The system was verified at four levels before and during the pilot, in keeping with the iterative methodology of Chapter 3:

Static verification. The entire server and mobile codebase is written in TypeScript and kept free of type errors; the Prisma ORM extends this guarantee to the database boundary, since queries are type-checked against the 31-model schema at compile time.

Component verification. Each pipeline stage was exercised against live data as it was added. The filtering thresholds in particular were tuned empirically: rejected fixes are logged with their rejection reason (Section 5.3), and reviewing those logs during early deployment confirmed that the gates rejected genuine noise—urban scatter, impossible jumps—without suppressing legitimate motion.

End-to-end verification on the road. The complete chain—GPS fix, cellular transmission, decoding, arbitration, filtering, ETA, push, and on-screen rendering—was verified with live test drives on the actual routes, observing the passenger application against the visible position of the real bus. Device-configuration changes were verified from two independent vantage points (Traccar's report view and the server's ingest log, Section 5.9).

Continuous operational verification. Because every layer logs (FR-13), the running pilot itself acted as a seven-week continuous test: anomalies surfaced in the audit data rather than in user reports, which is also how the idle-reporting problem of Section 6.5 was found.

6.2 Pilot Deployment and Dataset

UniBus Live was evaluated in a seven-week pilot, from 6 May to 23 June 2026, on the operational university fleet—real buses, real routes, real riders, zero simulation. The

deployment covered 9 routes and 3 buses, one of which was equipped with the fixed hardware tracker; 38 passenger accounts were registered, and 71 completed trips were recorded, during which 46,548 raw GPS reports were ingested. Table 6.1 summarizes the dataset.

Table 6.1: Pilot deployment summary.

Quantity	**Value**
Deployment span	7 weeks (6 May – 23 June 2026)
Routes	9
Buses	3 (1 fitted with hardware tracker)
Registered passengers	38
Completed trips	71
Raw GPS reports ingested	46,548
In-trip fixes retained	864
Mean GPS fixes per trip	37.6

One number in Table 6.1 demands immediate attention: of 46,548 raw reports, only 864 were retained as in-trip fixes. The remaining $\sim$98% were transmitted while buses were parked. This single observation—invisible during design, unmistakable in the operational data—motivated the efficiency optimization analysed in Section 6.5, and is in our view one of the most instructive findings of the whole project.

6.3 Measurement Methodology

All metrics are computed from the production database; no separate test harness or synthetic workload was used. Position-acquisition latency is the difference between each fix's device-reported timestamp and its server-receipt timestamp, both recorded at ingestion (Section 5.2). Source usage is taken from the canonical source selected per trip. Passenger engagement is measured from the notification delivery and read records. ETA accuracy is assessed through the prediction log: every predicted arrival instant is retained so it can be compared against the corresponding recorded stop arrival. Because the pilot is small, we report results as a feasibility demonstration rather than as population statistics.

6.4 Results

6.4.1 Position-Acquisition Latency

Across 46,286 fixes with valid timestamp pairs, the latency from device fix to server receipt was:

Table 6.2: Position-acquisition latency (device fix to server receipt).

Statistic	Latency
Median	5.9 s
Mean	7.4 s
90th percentile	9.9 s
95th percentile	11.1 s

A further median 1.2 s elapsed between server receipt and persistence. The latency distribution is bounded primarily by the 8 s Traccar polling interval rather than by network transport: when the WebSocket push path is active a fix arrives within about a second, and the polling path fills the remainder of the distribution. For the passenger this means the map reflects reality within roughly six seconds in the median case—comfortably inside the "live" perception window for a vehicle that takes minutes between stops (NFR-1 satisfied).

6.4.2 Source Resilience

The dual-source design was exercised in earnest rather than remaining a theoretical safeguard. Of the 70 trips with a recorded canonical source, 45 (64%) were tracked via the fixed hardware tracker and 25 (36%) via the smartphone fallback. The fallback share largely reflects the two buses not yet fitted with trackers—exactly the deployment reality the fallback was designed for. Coverage was sustained across the whole fleet throughout the pilot even though only one bus carried hardware.

Two readings follow. First, graceful degradation works: a bus without a tracker remained trackable, and mid-trip handovers produced no passenger-visible artefacts. Second, the design remains hardware-first: the tracker operates with no driver involvement, and as trackers are fitted fleet-wide the smartphone path reduces to a rarely needed safety net (NFR-2 satisfied).

6.4.3 ETA Estimation Behaviour

The geometric estimator produced a per-stop arrival prediction on every accepted fix, each carrying its HIGH/MEDIUM/LOW confidence label. Every prediction was logged against its trip and stop, so mean absolute error against recorded stop arrivals can be measured cumulatively as operational data grows. Within the pilot itself the prediction log was validated end-to-end (predictions recorded, arrivals recorded, joinable per stop); a statistically meaningful MAE requires more arrivals per stop than seven weeks provided, and is deliberately reported as accumulating future evidence rather than overstated from sparse data.

6.4.4 Passenger Engagement

Of 45 arrival and service notifications delivered to riders during the pilot, 36 (80%) were read. For a passive daily-utility application this read rate indicates that the alerts were genuinely attended to rather than dismissed as noise, supporting the design decision to invest in proximity-based drop-off alerts (FR-9).

6.5 The Idle-Reporting Problem and Optimization

This section analyses the pilot's most consequential finding.

6.5.1 The Discovery

The fixed-interval tracker transmitted a fix every 5 seconds whenever it was powered—and it is wired to the vehicle battery, so it was always powered. Buses, however, spend most of their day parked. The result, in plain numbers: of 46,548 raw reports, only 864 fell inside any active trip. Roughly **98% of all transmissions were waste**—consuming cellular data on the tracker's SIM, drawing on the bus battery, and occupying server ingestion for positions nobody would ever look at.

Nothing in the design phase predicted this, because each individual report is cheap; only the operational aggregate exposed the cost. This is precisely the kind of finding that distinguishes a deployed evaluation from a simulation.

6.5.2 The Two-Sided Fix

The waste was attacked at both ends of the pipeline:

- **Device side.** The tracker was reconfigured over SMS (`TIMER,5,300#`, Section 5.9) to an asymmetric cadence: every 5 s while moving, every 300 s while

static—a 60× reduction in parked-state transmissions at the source, saving cellular data and battery on the vehicle itself.

- **Server side.** The Traccar polling job was made trip-aware: full-rate 8 s polling now applies only to buses in an active trip or pre-trip window, while idle buses are polled every 60 s—a 7.5× reduction in idle polling, leaving the WebSocket push path and on-trip tracking untouched.

Table 6.3: Idle-reporting efficiency optimization.

Layer	Parameter	Before	After	Reduction
Device (GT06S)	Parked report interval	5 s	300 s	60×
Server	Idle poll interval	8 s	60 s	7.5×

6.5.3 Verification

The device-side change was verified from two independent vantage points: the per-position timestamps in Traccar's report view, and the server's own ingest log. Both showed parked-state fixes spaced by minutes rather than seconds after the change, while a live test drive confirmed that motion still produced dense five-second fixes—that is, the optimization removed only the redundant reports, not the live tracking (NFR-4 satisfied). After the optimization the server ingests essentially only the dense fixes of active trips plus a sparse five-minute parked heartbeat per bus, in place of the previous continuous five-second stream.

6.6 Requirements Coverage

Table 6.4 closes the loop between Chapter 3 and the pilot.

Table 6.4: Requirements coverage demonstrated in the pilot.

Req.	Pilot evidence
FR-1–FR-3	64%/36% tracker/fallback split across 70 sourced trips; mid-trip handovers without artefacts.
FR-4	Rejected fixes logged with reasons; displayed positions stable throughout.
FR-5	Per-stop ETA with confidence on every accepted fix; all predictions logged.
FR-6	Median 5.9 s acquisition latency; push delivery on every accepted update.
FR-7, FR-8	Pre-trip vehicle state shown before departure; staleness flagged within 30 s.
FR-9	45 notifications delivered; 80% read.
FR-10	All pilot riders used the bilingual interface; both scripts rendered.
FR-11–FR-12	9 routes, 3 buses, and all schedules administered through the console.
FR-13	Entire evaluation computed from production logs alone.
NFR-1–NFR-7	Addressed respectively in Sections 6.4.1, 6.4.2, 5.10, 6.5, 4.10, 4.9, and 5.6.

6.7 Limitations

The results must be read with the pilot's boundaries in mind. The scale is small—seven weeks, 3 buses, 38 riders—and a subset of trips were operational tests. Only one bus carried the hardware tracker, so the 64/36 source split reflects fleet hardware coverage as much as tracker reliability. The GT06S does not report a per-fix accuracy value, so quality filtering is evaluated at the trip level rather than per fix. Schedule-adherence analysis was limited by incomplete timetable association during the pilot. The findings therefore demonstrate feasibility; they do not claim city-scale statistical generality.

CHAPTER 7

Conclusion and Future Work

7.1 Conclusion

This thesis set out to answer a practical question: can a university in a developing region give its students live, trustworthy, in-their-own- language information about their bus, at a cost the institution can actually sustain? The answer, demonstrated on a real fleet rather than in simulation, is yes.

We designed, implemented, and deployed UniBus Live, a complete real-time bus tracking platform comprising a bilingual passenger mobile application, a web administration console, and a central application server backed by a 31-model relational schema—all self-hosted on a single low-cost server. Its distinguishing technical ideas are:

- **Resilient dual-source acquisition.** A fixed, vehicle-grade GPS tracker provides fully automatic, driver-independent positioning as the primary source, while the driver's smartphone serves only as an optional fallback, selected per fix by health-based arbitration. In the pilot this was no theoretical safeguard: 64% of trips ran on hardware and 36% on the fallback, sustaining coverage while most of the fleet awaited tracker installation.
- **A training-free geometric ETA estimator.** Projecting the live position onto the route polyline and dividing remaining distance by a rolling-average speed yields a per-stop arrival estimate with a confidence label from the first day of deployment, with every prediction logged for future learning-based refinement.
- **An edge-condition-aware passenger experience.** Pre-trip vehicle awareness, explicit staleness signalling, offline retention, and proximity-based drop-off alerts keep the interface honest and informative precisely where naive trackers show a blank or frozen map—in both English and Bangla.

The seven-week pilot on the operational fleet (9 routes, 3 buses, 38 riders, 71 trips, 46,548 GPS reports) established feasibility with a median position-acquisition latency of 5.9 s and an 80% read rate on delivered notifications. Just as importantly, the pilot

taught a lesson that no simulation would have: roughly 98% of raw device reports were redundant idle-state transmissions. The resulting two-sided optimization—asymmetric device reporting (5 s moving / 300 s parked) and trip-aware server polling (8 s active / 60 s idle)—cut idle-period reporting by up to 60× while leaving live tracking untouched, and stands as a concrete, transferable design guideline for any scheduled-fleet tracking deployment: *couple reporting cadence to vehicle state and timetable, at both the device and the server.*

7.2 Future Work

The deployed system and its accumulating operational data open several concrete directions:

1. **Learning-based ETA prediction.** Every prediction and every arrival is already logged. Once the trajectory history is large enough, a data-driven model (Section 2.4) can be trained and evaluated directly against the geometric baseline on the same logged data.
2. **Fleet-wide tracker installation.** Fitting the remaining buses reduces the smartphone path to a rarely needed safety net and enables uniform fleet analytics.
3. **Ignition-coupled reporting.** Wiring the tracker's ACC line would let the device sleep with the engine, completing the efficiency work of Section 6.5 with engine-state-triggered reporting and distance-based thresholds.
4. **Full timetable integration.** Associating every trip with its scheduled departure enables systematic schedule-adherence analytics for the transport office—the administrative counterpart of the passenger-facing ETAs.
5. **Crowd-sourced occupancy.** At larger scale, passenger participation could add "how full is the bus" to "where is the bus," complementing rather than carrying the positioning system.
6. **A formal usability study.** A wider, longer deployment with a structured passenger study would quantify the experience benefits that the pilot's 80% notification read rate only hints at.

7.3 Closing Remark

The deepest lesson of this project is methodological. The system's most valuable finding—the 98% idle-reporting waste—was not visible in any design document, simulator, or code review. It appeared only in the operational data of a real deployment, and once visible, it was fixable within days. Building the audit layer first, deploying early,

and letting production data drive the engineering agenda turned a student project into an evaluated system; we commend the same path to those who follow.

References

[1] S. A. Sharif, M. S. A. Suhaimi, N. N. Jamal, I. K. Riadz, I. F. Amran, and D. N. A. Jawawi, "Real-time campus university bus tracking mobile application," in *Proc. 7th ICT International Student Project Conference (ICT-ISPC)*, 2018.

[2] K. Premkumar, K. Pavithra, J. Presiela, D. Priyadarshini, and P. Priyanga, "College bus tracking and notification system," in *Proc. International Conference on System, Computation, Automation and Networking (ICSCAN)*, 2020.

[3] P. Zhou, Y. Zheng, Z. Li, M. Li, and G. Shen, "How long to wait? Predicting bus arrival time with mobile phone based participatory sensing," in *Proc. 10th ACM International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2012.

[4] P. Wepulanon, A. Sumalee, and W. H. K. Lam, "A real-time bus arrival time information system using crowdsourced smartphone data: a novel framework and simulation experiments," *Transportmetrica B: Transport Dynamics*, vol. 6, no. 1, 2018.

[5] T. Reich, M. Budka, D. Robbins, and D. Hulbert, "Survey of ETA prediction methods in public transport networks," *arXiv preprint arXiv:1904.05037*, 2019.

[6] N. Singh and K. Kumar, "A review of bus arrival time prediction using artificial intelligence," *WIREs Data Mining and Knowledge Discovery*, vol. 12, no. 4, p. e1457, 2022.

[7] L. Ye and P. Thiengburanathum, "A real-time bus arrival time prediction system based on Spark framework and machine learning approaches: a case study in Chiang Mai," in *Proc. Joint International Conference on Digital Arts, Media and Technology (ECTI DAMT & NCON)*, 2020.

[8] N. Rashvand, S. S. Hosseini, M. Azarbayjani, and H. Tabkhi, "Real-time bus arrival prediction: a deep learning approach for enhanced urban mobility," *arXiv preprint arXiv:2303.15495*, 2023.

[9] N. Rashvand, M. Azarbayjani, S. S. Hosseini, and H. Tabkhi, "Real-time bus departure prediction using neural networks for smart IoT public bus transit," *IoT*, vol. 5, no. 4, pp. 650–665, 2024.

[10] S. Jeong, C. Oh, and J. Jeong, “BAT-Transformer: prediction of bus arrival time with transformer encoder for smart public transportation system,” *Applied Sciences*, vol. 14, no. 20, art. 9488, 2024.

[11] R. Barnes, S. Buthpitiya, J. Cook, A. Fabrikant, A. Tomkins, and F. Xu, “BusTr: Predicting bus travel times from real-time traffic,” in *Proc. 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2020, pp. 3243–3251.

[12] P. Newson and J. Krumm, “Hidden Markov map matching through noise and sparseness,” in *Proc. 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, 2009, pp. 336–343.

[13] Traccar, “Traccar: Open source GPS tracking platform.” [Online]. Available: `https://www.traccar.org/`

[14] OpenJS Foundation, “Node.js.” [Online]. Available: `https://nodejs.org/`

[15] OpenJS Foundation, “Express: Fast, unopinionated, minimalist web framework for Node.js.” [Online]. Available: `https://expressjs.com/`

[16] Prisma Data, Inc., “Prisma ORM.” [Online]. Available: `https://www.prisma.io/`

[17] PostgreSQL Global Development Group, “PostgreSQL: The world’s most advanced open source relational database.” [Online]. Available: `https://www.postgresql.org/`

[18] Redis Ltd., “Redis.” [Online]. Available: `https://redis.io/`

[19] Socket.IO, “Socket.IO: Bidirectional and low-latency communication for every platform.” [Online]. Available: `https://socket.io/`

[20] Meta Platforms, Inc., “React Native: Learn once, write anywhere.” [Online]. Available: `https://reactnative.dev/`

[21] Expo, “Expo: Create universal native apps with React.” [Online]. Available: `https://expo.dev/`

[22] Vercel, Inc., “Next.js: The React framework.” [Online]. Available: `https://nextjs.org/`

[23] E. D. Kaplan and C. J. Hegarty, Eds., *Understanding GPS/GNSS: Principles and Applications*, 3rd ed. Norwood, MA, USA: Artech House, 2017.

[24] I. Fette and A. Melnikov, “The WebSocket Protocol,” IETF RFC 6455, Dec. 2011.

APPENDIX A

Tracker SMS Configuration Commands

The Concox GT06S tracker is configured entirely over SMS; each command is acknowledged by a return SMS from the device. Table A.1 lists the commands used during deployment and the pilot.

Table A.1: GT06S SMS commands used in this work.

Command	Effect
`TIMER,5,300#`	Set asymmetric reporting: every 5 s while the vehicle is moving, every 300 s while static. This is the device-side half of the idle-efficiency optimization (Section 6.5).
`PARAM#`	Read back the active configuration, used to verify that the `TIMER` change was applied.
`STATUS#`	Report battery level and GSM/GPS state, used for routine health checks during the pilot.

The device also emits a low-frequency heartbeat between position reports, which keeps it visible as *online* to the telematics server while parked.

APPENDIX B

Operator-Configurable Processing Parameters

Table B.1 consolidates the tunable parameters of the position-processing pipeline with the values used in the pilot. All are operator-configurable without code changes.

Table B.1: Pipeline parameters and pilot values.

Parameter	**Pilot value**	**Role**
Accuracy soft-reject	120 m	Quality gate (Section 4.5)
Accuracy hard-reject	250 m	Quality gate
Teleport guard	120 km/h	Physical plausibility
Minimum movement	4 m	Stationary-jitter floor
Reliable-speed threshold	5 km/h	ETA effective speed (Section 4.6)
Default speed	20 km/h	ETA fallback speed
Arrival radius	80 m	Stop-reached detection, alerts
Staleness check period	30 s	Trip staleness job
Active-trip poll interval	8 s	Traccar reconciliation
Idle poll interval	60 s	Trip-aware throttling
Moving report interval	5 s	Device cadence (GT06S)
Parked report interval	300 s	Device cadence (GT06S)

APPENDIX C

API Surface: Functional Overview

The application server exposes one REST API consumed by both clients, plus the Socket.IO realtime gateway. Table C.1 summarizes the API at the level of functional resource areas, indicating which client consumes each area. (Exact endpoint paths are an implementation detail and are omitted here; the table records the functional surface.)

Table C.1: Functional overview of the API surface.

Resource area	Operations	Consumer
Authentication	Registration, login, session management, OTP verification	Both clients
Routes & stops	Query route geometry, stop lists; create and edit routes (route builder)	Both / Admin
Schedules	Query today's and upcoming schedules; manage timetable entries	Both / Admin
Trips & live state	Query active trip, live tracking state, and per-stop ETAs	Passenger
Driver location	Publish authenticated fallback position reports	Driver mode
Favourites & alerts	Manage favourite stops and drop-off alert subscriptions	Passenger
Notifications	Deliver and mark-read arrival/service notifications	Passenger
Fleet & devices	Manage buses, GPS devices, and driver assignments	Admin
Users & roles	Manage accounts and role-based permissions	Admin
Realtime (Socket.IO)	Join per-trip rooms; receive position and ETA push events	Passenger / Admin

APPENDIX D

Passenger Application Screens

This appendix reproduces screens of the deployed passenger application, captured on an Android device during operation on the real routes. Figure D.1 shows the core tracking experience; Figure D.2 shows the alerting and engagement surfaces, including the honest edge-condition states discussed in Section 5.6.

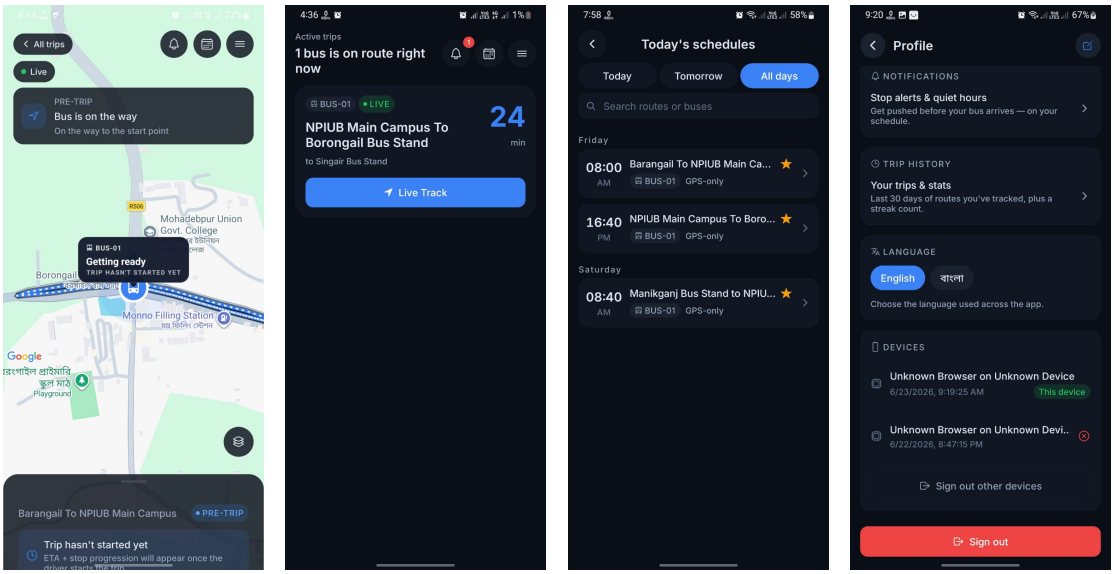


Figure D.1: Core tracking experience. Left to right: (a) the live map during the PRE_TRIP window—the bus is rendered at its pre-departure position with a "getting ready" state, so the map is never blank before departure; (b) the active-trips view showing a live bus (BUS-01) on route with its ETA to the rider's stop and a one-tap *Live Track* action; (c) today's schedules grouped by day and route; (d) the in-app language setting with the English/Bangla toggle.

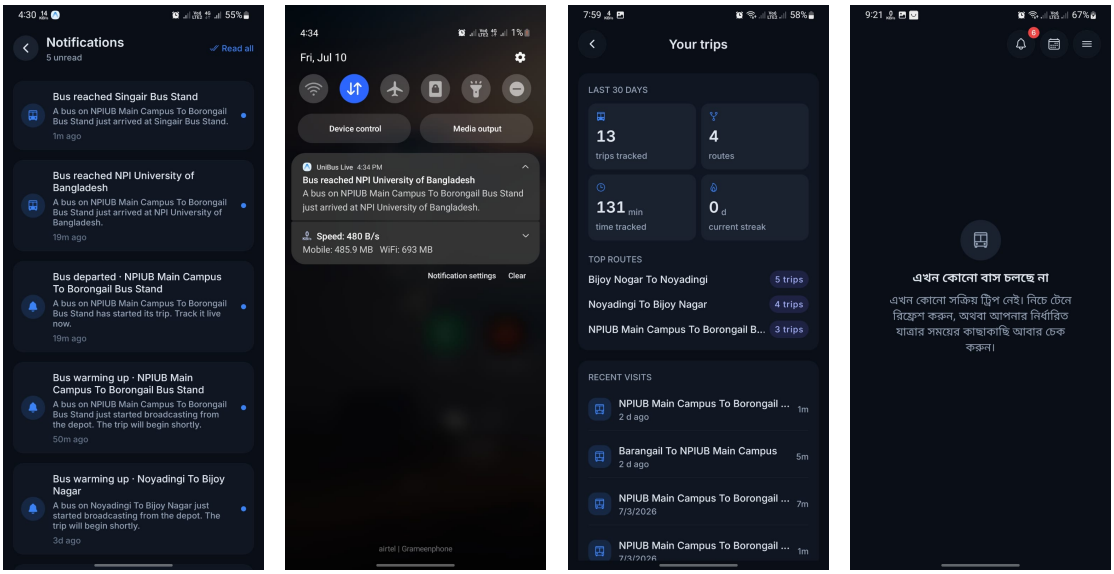


Figure D.2: Alerting and engagement surfaces. Left to right: (a) the in-app notification feed recording trip events (bus warming up, departed, reached a stop); (b) a system push notification delivered outside the app announcing that the bus reached the campus; (c) the rider's trip history and statistics (trips tracked, routes used, time tracked); (d) the Bangla-localized empty state shown when no bus is currently running—an example of the fully bilingual interface (FR-10).

APPENDIX E

Administration Console Screens

This appendix reproduces the management pages of the web administration console that complement the principal screens shown in Figure 5.3. All screens were captured from the deployed console operating on the real fleet data.

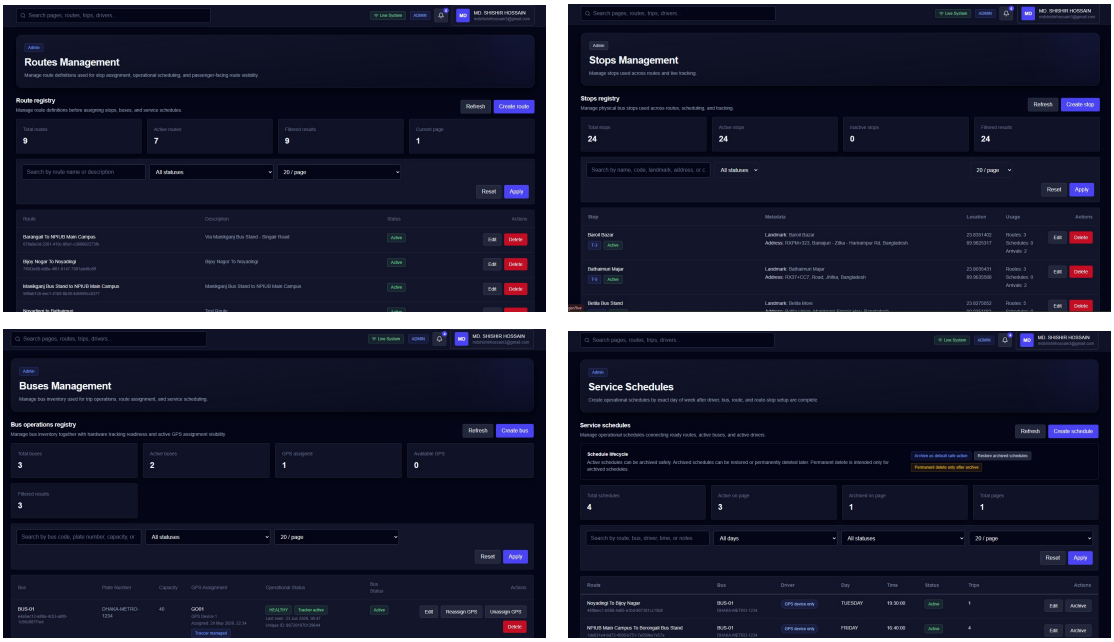


Figure E.1: Management pages of the administration console. Top left: the route registry with per-route status and stop-visibility settings. Top right: stops management, maintaining the physical bus stops used across routes. Bottom left: bus operations registry, pairing each bus with its hardware tracking readiness and GPS assignment. Bottom right: service schedules, binding routes, buses, drivers, days, and departure times.

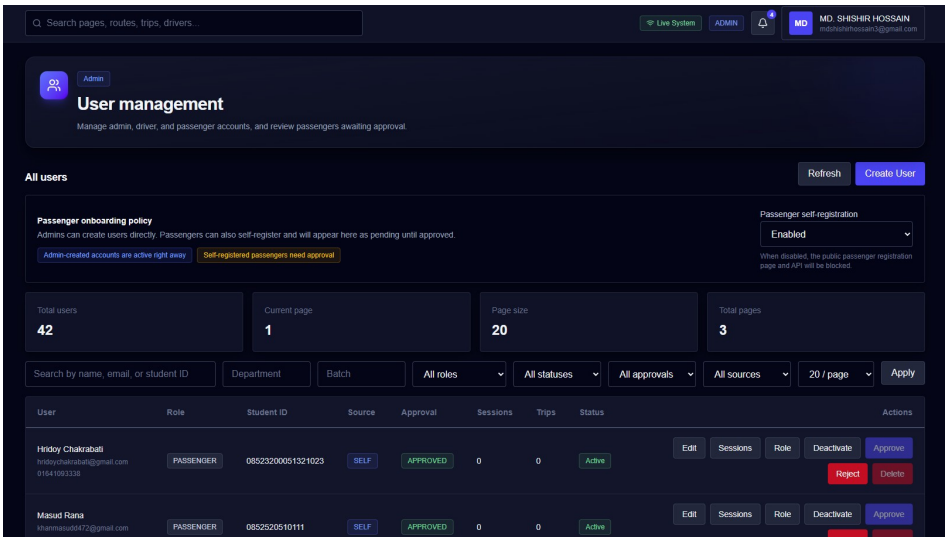


Figure E.2: The user management page: accounts, roles, per-user sessions, and the passenger onboarding approval policy (NFR-5).